MarginGraph: Margin-Aware, Constrained Recommendation of Checkout Add-Ons

A reproducible study on UCI Online Retail II, with a negative result for small LLM rerankers

Sanskar
github.com/Sanskar1724/MarginGraph

Draft of 26 October 2026

Abstract

Recommenders in retail are usually trained to predict what a shopper will buy, while the store is paid by what it earns. We study this gap for a narrow, common decision: which (at most two) add-on products to suggest at checkout. MarginGraph is a two-stage system. Stage 1 retrieves 150 candidates from eleven sources (classical recommenders, text similarity and a nightly fresh-sales signal) and reranks them with a LightGBM LambdaRank model whose scores are calibrated into purchase probabilities. Stage 2 solves a small mixed-integer program per basket that trades expected relevance against expected gross profit, under a hard 15% per-item margin floor, a two-item limit, stock availability, and an optional overstock weight. On a strict temporal split of UCI Online Retail II (4,480 test baskets), the reranker reaches Hit@10 of 38.1%, 7.0 points above the best baseline (EASE, 31.1%; paired 95% CI [6.0, 8.0]). With *simulated* unit cost and stock, the optimizer at λ=0.5 raises realized gross profit per basket by 14% over the ranker's top-2 (paired CI of the gain £0.014–0.044) for a 1.1-point drop in Hit@2, with zero margin-floor violations; the gain stays positive over nine cost and stock scenarios. We also report a negative result: zero-shot Qwen2.5 1.5B and 3B rerankers, their rank blends, a stacker that feeds the LLM score to a logistic model, and a LoRA fine-tune of the 1.5B model all fail to beat LightGBM on the same slates. All numbers in this paper are read from files the pipeline writes; none are targets.

1 Introduction

A checkout add-on is a small, high-volume decision. The shopper has already chosen a basket; the store may show one or two more products. Showing the product the shopper is most likely to add maximises conversions, but that product may carry a thin margin, may be out of stock, or may compete with stock the store needs to clear. Price- and profit-aware recommendation is a long-standing topic [1, 13], yet most open recommender benchmarks report only accuracy, and many published accuracy gains do not survive strong, well-tuned baselines [8].

This paper makes four contributions.

1. **A leakage-free checkout add-on benchmark** on UCI Online Retail II [5]: one product hidden per invoice, one global temporal cutoff per split, and full-catalog ranking (no sampled negatives [15]).

2. **A strong Stage 1**: a baseline ladder of eleven candidate sources, a 150-item candidate pool, and a calibrated LightGBM reranker, each step measured with bootstrap confidence intervals.

3. **A Stage 2 optimizer** that turns calibrated probabilities into an add-on choice under hard business rules, checked against exact enumeration on 600 problems, evaluated by replaying real purchases, and stress-tested across simulated cost and stock scenarios.

4. **A documented negative result** for small open LLMs as rerankers on this task, including a blend and a stacker, run on the same slates as LightGBM.

The dataset has prices but no cost or stock, so every profit number depends on a simulation that we state in full (Section 4.3). We treat this as the main limitation of the study rather than a footnote.

2 Related work

Classical next-item and basket recommenders. Item-based collaborative filtering [20], implicit-feedback matrix factorization (ALS) [11] and the closed-form linear autoencoder EASE [21] remain strong on sparse purchase data. Ferrari Dacrema et al. [8] showed that simple, tuned neighbourhood methods often match or beat recent neural models, which is why our ladder includes them and why the reranker must beat the best of them on paired tests. Krichene and Rendle [15] showed that sampled ranking metrics can reorder models; we therefore rank against the full catalog.

Learning to rank and calibration. We rerank with LambdaRank-style gradient boosting [4] as implemented in LightGBM [14]. Stage 2 needs probabilities, not scores, so we fit an isotonic map from score to purchase probability [22].

Profit-aware and multi-objective recommendation. Jannach and Adomavicius [13] survey price and profit awareness in recommenders, and Abdollahpouri et al. [1] frame the store as one of several stakeholders whose utility a recommender should account for. Louca et al. [16] jointly optimise profit and relevance in e-commerce ranking at Etsy; Pei et al. [18] maximise expected profit with reinforcement learning at Taobao; Agarwal et al. [2] optimise one objective under constraints on others for content recommendation. MarginGraph is closest to the constrained view: it keeps relevance in the objective and puts the store's rules (margin floor, item limit, stock) in hard constraints, solved exactly per basket with an open-source MILP stack [12, 17]. Unlike the industrial studies above, every component and result here is public and reproducible on a free dataset and a CPU.

LLMs as recommenders. Hou et al. [9] show that LLMs can rank candidates zero-shot but are sensitive to order and history length and trail trained recommenders on several benchmarks. TALLRec [3] shows that parameter-efficient tuning (LoRA [10]) aligns an LLM with recommendation data far better than prompting. We test the cheapest version of this idea a student can run for free: Qwen2.5 [19] 1.5B and 3B on a single Kaggle T4 GPU.

3 Data and task

Data. UCI Online Retail II [5, 6] contains the transactions of a UK online gift and homeware wholesaler from December 2009 to December 2011. From 1,067,371 raw rows we drop cancellations and adjustments (19,500), non-positive quantity or price (6,202), non-product codes such as postage and fees (4,792), and exact duplicates (33,663), leaving 991,910 invoice lines over 39,516 invoices, 4,707 products and 5,852 known customers. The data has a unit price but no unit cost and no stock level.

Task. For each evaluation invoice with 2–50 distinct products we hide one product at random; the model sees the remaining basket (and the customer's earlier history, if known) and must rank the full catalog. The hidden product is the single relevant item. We report Hit@K, NDCG@10 and MRR.

Split. One global cutoff per boundary: train up to 2011-06-30 (709,574 lines), validation July–September 2011 (3,822 events), test 2011-10-02 to 2011-12-09 (4,480 events), with no invoice in two splits. Test candidates come from models refit on train+validation, which ends before the test window starts. 7.1% of test targets never appear before the test window, so no model can exceed 92.9% Hit@K on test.

Signal audit (E1). Before modelling we checked that baskets carry a learnable signal. On validation, a co-purchase model fit on real baskets reaches Hit@10 of 16.1%, against 4.4% when the same model is fit on baskets whose items were shuffled across invoices, and 0.23% for chance. The highest-lift pairs are recognisable complements (matching paper cups and plates; egg cosies in two colours).

4 Method

4.1 Stage 1: retrieval, reranking and calibration

Baselines and candidate pool. Seven classical scorers are fit on the training period: global popularity, 90-day popularity, the customer's repeat purchases, co-purchase counts, ItemKNN [20], EASE (λ=250, chosen on validation) [21] and ALS (64 factors) [11]. Four further sources were added after a validation-only selection (Section 5.1): EASE on recency-weighted baskets (half-life 120 days), EASE applied to the customer's own past items, TF-IDF similarity of product descriptions (word 1–2 and character 3–4 grams, maximum over the basket), and a *fresh* signal that, like a nightly job, sees only invoices from days before the event: 14-day trending items and co-purchase over that stream. The item index covers the whole catalog, so products first sold after the fit cutoff can be recommended once the fresh source sees them sell; a product with no earlier sales gets zero from every source. The candidate pool unions the top lists of each source in a fixed order (EASE 90, decayed EASE 60, customer EASE 30, text 30, trending 22, fresh co-purchase 30, ItemKNN 45, ALS 45, repeat 45, co-purchase 30, the two popularity scorers 15 each), deduplicated and capped at 150 items, with basket items removed.

Reranker. A LightGBM LambdaRank model [4, 14] (31 leaves, learning rate 0.05, up to 500 trees with early stopping) scores each candidate from 35 features: the score and rank from every source, price, price relative to the basket, item age and recency, basket size, share of basket items in the same category, and whether the customer is known. It is trained on validation events whose candidates were produced by train-only models, so the reranker never sees test-period data; each validation invoice contributes three events, each hiding a different product (early stopping and calibration use only the original one). The most important features by gain are the decayed-EASE and EASE ranks, the repeat-purchase score and the text similarity.

Calibration. An isotonic regression [22], fit on a held-out, later slice of validation events, maps the reranker score to p_i, the probability that candidate i is the product the shopper adds.

4.2 Stage 2: constrained add-on selection

For a basket, take the top N=20 reranked candidates. Let $x_{i,d} \in \{0,1\}$ mean "offer item i at discount d", with price r_i, simulated unit cost c_i, revenue $r_{i,d} = r_i(1-d)$ and unit profit $\pi_{i,d} = r_{i,d} - c_i$. The optimizer solves

$$\max_{x} \sum_{i,d} x_{i,d}\, p_{i,d} \left[(1-\lambda)\,\bar{\pi} + \lambda\, w_i\, \pi_{i,d} \right] \tag{1}$$

$$\text{s.t. } \textstyle\sum_d x_{i,d} \le 1 \;\; \forall i, \qquad \sum_{i,d} x_{i,d} \le 2,$$

$$x_{i,d} = 0 \text{ if stock}_i \le 0, \qquad \pi_{i,d} \ge 0.15\, r_{i,d} \text{ whenever } x_{i,d} = 1.$$

Table 1: Full-catalog ranking on 4,480 test baskets. Best baseline in italics. LightGBM minus EASE, paired: Hit@10 +7.0 pts [6.0, 8.0], NDCG@10 +0.049 [0.043, 0.055].

Model	Hit@5	Hit@10	Hit@50	NDCG@10	MRR
Popularity	1.9	3.1	10.0	0.016	0.017
Recent popularity (90 days)	1.8	3.0	10.6	0.016	0.018
Trending, 14 days (fresh)	3.1	5.4	16.5	0.029	0.029
Customer EASE	6.0	9.5	22.7	0.050	0.045
Co-purchase	9.2	13.0	24.8	0.079	0.071
Repeat purchase	9.7	13.4	26.9	0.078	0.068
Fresh co-purchase	12.4	16.8	29.8	0.104	0.094
ALS	12.7	17.3	31.8	0.105	0.094
Text similarity (TF-IDF)	14.3	20.5	39.4	0.113	0.096
ItemKNN	17.4	22.7	36.6	0.144	0.128
Decayed EASE	22.9	30.1	48.7	0.185	0.160
EASE	*23.2*	*31.1*	*48.9*	*0.188*	*0.161*
LightGBM rerank, first run (pool 100)	26.1	35.6	54.6	0.217	0.184
LightGBM rerank	**28.8**	**38.1**	**58.4**	**0.237**	**0.204**

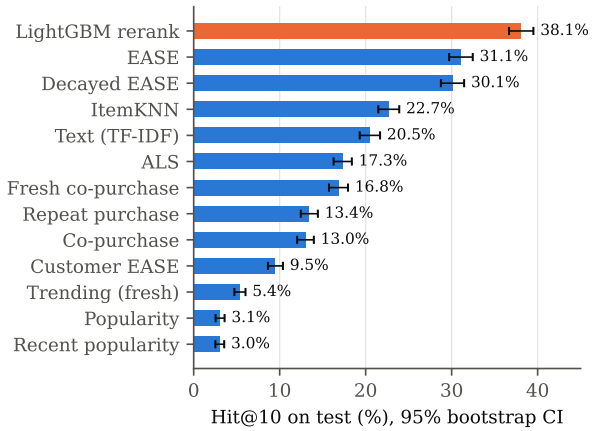
Here $\bar{\pi}$ is the catalog's mean unit gross profit, which puts the relevance term on the same scale as profit, so λ=0 ranks by probability alone and λ=1 maximises overstock-weighted expected profit. $w_i \in [1,2]$ up-weights items flagged as overstock. We also test a bundle-level margin floor in place of the per-item floor. Discounts use an *assumed* response $p_{i,d} = \min(1,\, p_i(1+1.5d))$ for $d \in \{0, 0.1, 0.2\}$; this elasticity is not estimated from data. The program is built in PuLP [17] and solved with HiGHS [12]; because at most two items are chosen, exact enumeration of all singles and pairs gives the same optimum and is used for the bulk offline sweeps.

4.3 Simulated cost and stock

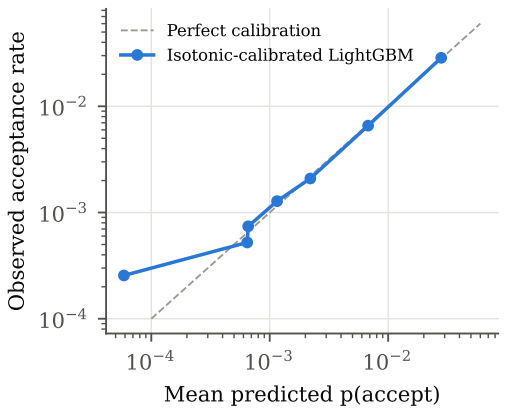
Unit cost is $c_i = r_i(1-m_i)$ with gross margin $m_i \sim \text{Beta}(4,6)$ (mean 40%). Base stock is six weeks of the item's mean weekly training demand; 15% of items are flagged overstock and given 3–6× that stock. Section 5.3 repeats the main comparison under margins drawn from Beta(2,8) (mean 20%) and Beta(6,4) (mean 60%), each with three random seeds.

4.4 Evaluation of Stage 2

Replay scoring counts an offer as accepted only if it is the product the shopper actually added (the hidden item); this is grounded in real behaviour and conservative. *Realized profit* is the unit gross profit of an accepted offer, averaged over all test baskets. *Expected profit* instead sums $p_{i,d}\,\pi_{i,d}$ over the offered items and so depends on calibration. For stock, a sequential simulation replays test baskets in time order against shared, decrementing stock. All confidence intervals are 95% bootstrap intervals over test baskets with 2,000 resamples [7]; differences between systems are paired over the same baskets.



(a) Baseline ladder and reranker, Hit@10 on test.



(b) Reliability of calibrated p_i on test (log scale; lowest bin omitted).

Figure 1: Stage 1 results.

5 Results

5.1 Stage 1

Table 1 and Figure 1a show the ladder. EASE is the strongest single model by a wide margin, consistent with Steck [21] and Ferrari Dacrema et al. [8]. The reranker improves on it on every metric, and the paired intervals exclude zero.

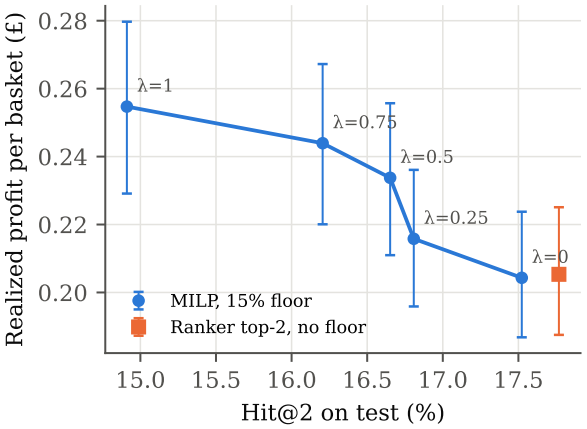
Retrieval ceiling. The 150-item pool contains the hidden product for 65.3% of test baskets [63.9, 66.7]; EASE alone reaches 55.7% at depth 100, decayed EASE 56.3%, text 48.7%, ItemKNN 43.4%. This recall caps every reranker, including an LLM: 35% of test baskets cannot be answered by any choice of Stage 1 order. Earlier configurations reached 43.5% (50 items, no EASE) and 58.0% (100 items, seven sources).

Calibration. On test, the calibrated probabilities have a Brier score of 0.00417 against 0.00435 for a constant base-rate guess (first run: 0.00560 against 0.00568). They track observed rates across bins (Figure 1b) and are sharper than in the first run, but still modest, which limits how much Stage 2 can separate good from bad offers.

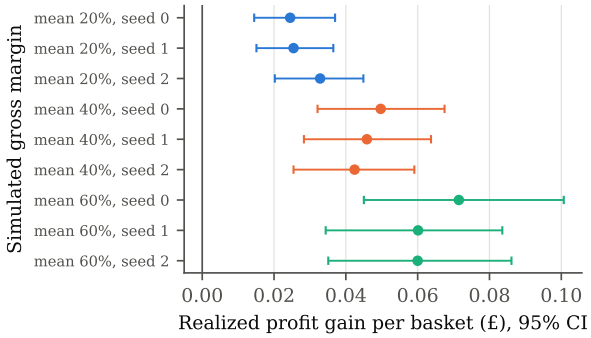
What raised Stage 1 (validation-only selection). The first run used seven sources, a 100-item pool and one event per validation invoice (Hit@10 35.6%). Each change was kept only if it won on the last 30% of validation events (1,147 events never used to fit trees); test results were recorded for all 19 variants, winners and losers. Added one at a time to the 100-item pool, each new source was within noise, because it displaced EASE and ItemKNN candidates. Together, with the pool enlarged to 150 (chosen over 100 and 200 on validation), they raised pool recall by 7.3 points [6.2, 8.3]; training on three events per validation invoice added the rest. Against the first run on the same test baskets, Hit@10 rose by 2.5 points [1.6, 3.4] and NDCG@10 by 0.020 [0.015, 0.025]. An item2vec source and four LightGBM parameter sets lost on validation and were dropped.

Table 2: Stage 2 on 4,480 test baskets, **simulated cost and stock**. Profit is £ gross profit per basket. Baselines run without the floor; the last column counts how often they would break the 15% per-item floor.

Strategy	Hit@2 (%)	Realized profit	Expected profit	Floor breaks / basket
Popularity top-2	1.1	0.013	0.014	0
Co-purchase top-2	6.1	0.072	0.057	0.004
Ranker top-2	17.8	0.205	0.185	0.037
MILP λ=0	17.5	0.204	0.186	0
MILP λ=0.5	16.7	0.234	0.212	0
MILP λ=1	14.9	0.255	0.223	0
MILP λ=1, overstock w=2	14.6	0.245	0.219	0
MILP λ=1, bundle floor	15.0	0.256	0.223	0
MILP λ=1, discounts 0/10/20%	14.9	0.255	0.223	0



(a) Relevance–profit frontier as λ varies.



(b) Gain of MILP λ=1 over ranker top-2 under nine cost/stock scenarios.

Figure 2: Stage 2 results (simulated cost and stock; 95% bootstrap CIs).

5.2 Stage 2

Table 2 and Figure 2a give the trade-off. Paired against the ranker's top-2 on the same baskets, MILP λ=0.5 raises realized profit by £0.028 per basket [0.014, 0.044], or 14%, while Hit@2 falls by 1.1 points [−1.7, −0.6]. At λ=1 the gain is £0.049 [0.031, 0.069] (24%) for −2.9 points of Hit@2 [−3.7, −2.0]. At λ=0 the optimizer is, as intended, almost identical to the ranker (−0.25 points Hit@2 and −£0.001 profit), the small gap coming from the margin floor removing some candidates. Every MILP solution satisfies every hard rule, while the unconstrained ranker top-2 would break the margin floor 0.037 times per basket. The stronger ranker also lifted the ranker's own top-2, from Hit@2 16.7% and £0.184 in the first run to 17.8% and £0.205, so the optimizer's relative gain is measured against a harder reference (first run: +17%).

Three further observations. (i) A bundle-level floor behaves like the per-item floor here. (ii) With discounts allowed, the optimizer almost never discounts (mean discount 0.07%): under the assumed elasticity, giving away margin rarely pays. (iii) The solver is not a bottleneck: PuLP with HiGHS and exact enumeration agree on all 600 checked problems (maximum objective gap 0), with a mean MILP solve time of 2.7 ms per basket on a CPU.

Table 3: LLM reranking on the top-20 LightGBM slate, 1,000 random test baskets, same slates for all rows. Differences are paired against LightGBM with 95% bootstrap CIs, in points. Kaggle T4 GPU.

System	Hit@5	Hit@10	Hit@5 − LightGBM
LightGBM	27.6	36.1	—
Qwen2.5-1.5B zero-shot	16.8	29.5	−10.8 [−13.7, −7.9]
rank blend with LightGBM	23.2	34.8	−4.4 [−6.5, −2.4]
Qwen2.5-3B zero-shot	16.6	—	−11.0 [−13.7, −8.4]
rank blend with LightGBM	22.3	33.9	−5.3 [−7.2, −3.4]
Stacker (LightGBM + 3B features)	27.5	—	−0.1 [−0.6, +0.3]
Qwen2.5-1.5B LoRA fine-tuned	7.8	17.4	−19.8 [−22.8, −16.7]
rank blend with LightGBM	16.2	30.4	−11.4 [−13.9, −8.8]
Random order of the slate	11.6	23.2	—

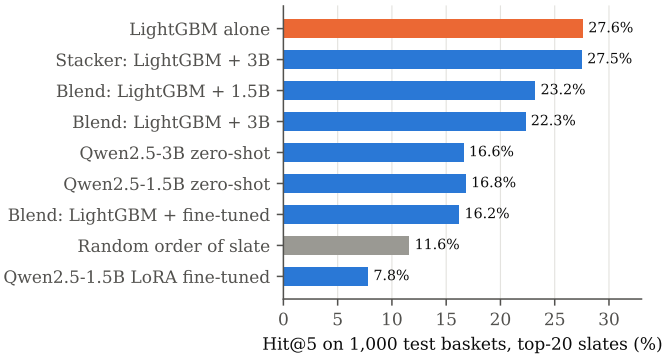


Figure 3: Hit@5 of LLM-based rerankers on the same 1,000 test slates.

Overstock (E10). In the sequential replay with shared stock, MILP $\lambda=1$ earns £1,141 over the test period against £920 for the ranker's top-2. Raising the overstock weight from 1.0 to 2.0 increases overstock units sold from 78 to 116, at a 3.7% profit cost (£1,099). The plain profit objective ($w=1$) sells fewer overstock units than the ranker (114); the weight is what moves stock, and at $w=2$ the optimizer roughly matches the ranker on overstock while earning 19% more. Over 20 stochastic acceptance seeds the ordering is the same (ranker £829, MILP $\lambda=1$ £993, $w=2$ £976 on average).

5.3 Sensitivity to the simulated cost

Figure 2b repeats the MILP $\lambda=1$ versus ranker comparison for three margin distributions and three random seeds each. The realized-profit gain stays positive in all nine scenarios (from £0.024 to £0.071 per basket) with every confidence interval above zero; the Hit@2 cost ranges from 2.3 to 6.6 points and is largest when margins are low, where the floor removes more candidates. This shows the direction of the result is not an artefact of one cost draw; it does not show its size on a real store's costs.

5.4 Small LLMs as rerankers: a negative result

We asked Qwen2.5-Instruct [19] a yes/no question for each (basket, candidate) pair ("Would this customer also add *X* to the same order?", with the basket's product descriptions) and ranked by the probability of "Yes". Table 3 and Figure 3 report the results. Both model sizes are far behind LightGBM, and the 3B model is no

better than the 1.5B model. Averaging LLM and LightGBM ranks still loses at Hit@5, and at Hit@10 the 1.5B blend is not significantly different (-1.3 [-3.0, $+0.4$]) while the 3B blend loses (-2.2 [-4.0, -0.4]). A logistic stacker fit on 1,147 held-out validation baskets, with LightGBM and 3B scores as inputs, ties LightGBM (-0.1 points at both Hit@5 and Hit@10) and gives the LLM features near-zero weight (0.015 and 0.055 against 0.736 for LightGBM, on standardised inputs). In short, zero-shot Qwen adds no measurable signal beyond the trained reranker on this task.

This agrees with Hou et al. [9], who find zero-shot LLM rankers trail trained models, and suggests the gap is larger for wholesale baskets described only by terse product names. We then fine-tuned the 1.5B model with LoRA [3, 10] on 10,772 training-period pairs (each hidden item with three LightGBM hard negatives from the same slate; two epochs, class-weighted yes/no loss). The weighted loss fell to about $\ln 2$ within 300 steps and stayed there, and the fine-tuned model is worse than zero-shot: Hit@5 7.8%, below the 11.6% of a random order of the same slate (the target is in the slate for 46.3% of baskets). Its blend with LightGBM also loses at Hit@10 (-5.7 [-7.7, -3.6]). A likely cause, which we did not test, is that LightGBM's near-misses are often the same product in another colour or size, which terse product names cannot separate. We therefore keep LightGBM as MarginGraph's ranker and limit the LLM to explanations and to products with no purchase history (7.1% of test targets); neither use is evaluated here.

6 Limitations

- **Simulated economics.** Unit cost, stock and the overstock flag are simulated, and the discount response is assumed. All Stage 2 profit numbers, including the 14% and 24% gains, are conditional on these assumptions. Section 5.3 varies the cost distribution, not its structure (for example, costs correlated with popularity).
- **Offline replay.** Replay credits an offer only if the shopper bought it without being shown it. It cannot see purchases an offer would have caused, so it likely understates any recommender, and it cannot measure how shoppers react to being shown higher-margin items. Only an online A/B test can.
- **One store, one task.** One UK wholesaler, mostly business buyers, 2009–2011. Hiding one item per invoice is a proxy for a real checkout widget.
- **Retrieval ceiling.** 35% of test targets are outside the candidate pool, which bounds every Stage 1 and Stage 2 result.
- **LLM scope.** We tested two small instruction-tuned models zero-shot, one prompt and one scoring method, on the slates of the first ranker (Hit@10 35.6%). One LoRA fine-tune of the 1.5B model with hard negatives also failed. Larger models, other prompts, richer product text, or easier negatives may behave differently.
- **Latency.** Solver time is 2.7 ms. The serving latency in the repository (p50 22 ms for the research API, p95 88 ms end to end for the multi-store service) was measured in a container, not in a store's production setting.

7 Conclusion

On a public retail dataset and a CPU, a calibrated gradient-boosted reranker beats a strong classical ladder for checkout add-ons, and a tiny per-basket MILP converts its probabilities into choices that respect hard margin and stock rules while raising (simulated) profit at a small, tunable cost in hit rate. Small open LLMs did not help, either zero-shot or after a LoRA fine-tune. The next steps are to replace simulated costs with a

real store's data, to run an online test, and to measure the LLM where it is still used: cold-start products and explanations.

Reproducibility

Code, configuration and every result file are in the repository. `margingraph run` rebuilds experiments E1–E11 on a CPU (seed 42) and writes them to `results/`; `paper/figures/make_figures.py` rebuilds the figures from those files. The LLM experiments use `notebooks/llm_rerank_gpu.ipynb` and `notebooks/llm_finetune_gpu.ipynb` on a free Kaggle T4 GPU; their numbers are recorded in `docs/RESULTS.md`.

Use of AI tools

The software, experiments and this manuscript were developed with the help of an AI coding assistant (Claude Code). All reported numbers come from the pipeline's output files or from logged Kaggle runs, and were checked against them.

References

[1] Himan Abdollahpouri, Gediminas Adomavicius, Robin Burke, Ido Guy, Dietmar Jannach, Toshihiro Kamishima, Jan Krasnodebski, and Luiz Pizzato. Multistakeholder recommendation: Survey and research directions. *User Modeling and User-Adapted Interaction*, 30:127–158, 2020. doi: 10.1007/s11257-019-09256-1. arXiv:1905.01986.

[2] Deepak Agarwal, Bee-Chung Chen, Pradheep Elango, and Xuanhui Wang. Click shaping to optimize multiple objectives. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2011. doi: 10.1145/2020408.2020435.

[3] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, 2023. arXiv:2305.00447.

[4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

[5] Daqing Chen. Online retail II. UCI Machine Learning Repository, 2019. Dataset 502, donated 2019-09-20.

[6] Daqing Chen, Sai Laing Sain, and Kun Guo. Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3):197–208, 2012.

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[8] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, 2019.

[9] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval (ECIR)*, 2024. arXiv:2305.08845.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[11] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *IEEE International Conference on Data Mining (ICDM)*, 2008.

[12] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[13] Dietmar Jannach and Gediminas Adomavicius. Price and profit awareness in recommender systems. *arXiv preprint arXiv:1707.08029*, 2017. Workshop on Value-Aware and Multistakeholder Recommendation (VAMS) at RecSys 2017.

[14] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[15] Walid Krichene and Steffen Rendle. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020. doi: 10.1145/3394486.3403226.

[16] Raphael Louca, Moumita Bhattacharya, Diane Hu, and Liangjie Hong. Joint optimization of profit and relevance for recommendation systems in e-commerce. In *Workshop on Recommendation in Multistakeholder Environments (RMSE) at ACM RecSys*, volume 2440 of *CEUR Workshop Proceedings*, 2019.

[17] Stuart Mitchell, Michael O'Sullivan, and Iain Dunning. PuLP: A linear programming toolkit for python. Optimization Online, 2011. The University of Auckland.

[18] Changhua Pei, Xinru Yang, Qing Cui, Xiao Lin, Fei Sun, Peng Jiang, Wenwu Ou, and Yongfeng Zhang. Value-aware recommendation based on reinforced profit maximization in e-commerce systems. *arXiv preprint arXiv:1902.00851*, 2019.

[19] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[20] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW)*, 2001.

[21] Harald Steck. Embarrassingly shallow autoencoders for sparse data. In *The World Wide Web Conference (WWW)*, pages 3251–3257, 2019. doi: 10.1145/3308558.3313710. arXiv:1905.03375.

[22] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2002.